

Reviewer Pack — Additive Energy Threshold and Deterministic Communication Co...

Entry ec6a507b1136 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 19:01:16 UTC

Additive Energy Threshold and Deterministic Communication Complexity

Entry ID: ec6a507b1136 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-07 19:01:16 UTC

1. Conjecture

Field A (mathematical branch): Additive combinatorics **Field B** (complexity object): Deterministic communication complexity

Statement:

For any Boolean function f on n variables, if the additive energy of its truth table $\Lambda(f) \geq n^3$, then the deterministic communication complexity of f is $\Omega(n)$.

Rationale (proposer's reasoning):

High additive energy implies a rich additive structure, which may necessitate high communication complexity due to the need for resolving conflicting additive relations between inputs.

Taxonomy category: ACC_SIPSER (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3daaf9478973204c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_additive_energy(truth_table):
        n = truth_table.index(1).bit_length() - 1
        energy = 0
        for x, y, z, w in combinations(range(2**n), 4):
            if (truth_table[x] + truth_table[y]) == (truth_table[z] + truth_table[w]):
                energy += 1
        return energy

    def deterministic_communication_complexity(truth_table):
        n = truth_table.index(1).bit_length() - 1
        instances_tested = 0
        for x in range(2**n):
            for y in range(2**n):
                if (truth_table[x] + truth_table[y]) == 1:
                    instances_tested += 1
        return instances_tested

    n = random.randint(5, 40)
    f = generate_boolean_function(n)
    energy = compute_additive_energy(f)

    communication_complexity = deterministic_communication_complexity(f)

    conjecture_holds = energy >= n**3 and communication_complexity >= n
    counterexample = "" if conjecture_holds else "energy < n^3 or communication complexity < n"

    return {
        "metric_name": "communication_complexity",
        "metric_value": communication_complexity,
        "instances_tested": 2**n * (2**n - 1),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
```

```

seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = (sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))**0.5
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value:.2f} std={std_value:.2f} support_fraction={support_fraction:.2f}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing validation of support/falsification criteria. | next: Run test with increased time limit and targeted parameter adjustments for critical cases

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	42089
2	propose	ollama_remote	qwen3:8b	0	0	30107
3	preregistration	ollama_remote	qwen3:8b	0	0	9047
4	novelty	ollama_remote	qwen3:8b	0	0	60059
5	novelty	ollama_remote	qwen3:8b	0	0	14332
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13322

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7644
8	judge	ollama_remote	qwen3:8b	0	0	67228

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 243828 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/ec6a507b1136.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/ec6a507b1136.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/ec6a507b1136.tar.gz (if generated)